N.Spandana
CDS/2025/1299

Q1) Reverse a string using stack

*algorithm

1. start
2. Read the string
3. push each character into stack
4. pop characters one by one
5. print popped characters [Reversed string]
6. stop

C program

```
# include <stdio.h>
# include <string.h>
# define MAX 100
    char stack[MAX];
      int top = -1;
    //push function
    void push (char ch){
        stack [++top]= ch;
      }
    //pop function
    char pop() {
          return stack [top--];
        }
int main() {
        ~~char pop() {~~
            ~~return stack[top--];~~
        char str [100];
        int i;
        Printf ("enter string: ");
        Scanf ("%s", str);
      // push all characters
      for (i=0; i<strlen(str); i++){
            push (str[i]);
```

```
}
Printf("reversed string:");
// pop to get reverse
while (top != -1) {
    Printf("%c", pop());
}
return 0;
}
```

output:

Enter string: DATA
Reversed string: ATAD

Q2: Balanced parentheses

algorithm

1. Start
2. Read expression
3. push opening brackets into stack
4. for closing brackets
   - check stack top
   - If matching → pop
   - Else → not balanced
5. If stack empty → balanced
6. stop

C program:

```
#include <stdio.h>
#include <string.h>
# define MAX 100
char stack [MAX];
int top = -1;
// push bracket
void push(char ch){
stack [++top] = ch;
}
// pop bracket
char pop() {
```

```
return stack [top--];
}
int in Matching pair(charopen, charclose){
  if (open == '(' && close == ')') return 1;
  if (open == '{' && close == '}') return 1;
  if (open == '[' && close == ']') return 1;
}
int main (){
    char exp[100];
     int i;
   Printf ("enter expression:");
    Scanf ("%s", exp);
  for (i=0; i<strlen(exp); i++){
  if (exp[i] == '(' || exp[i] = '{' || exp[i] == '[']
       push (exp[i]);
   else if (exp[i] == ')' || exp[i] == '}' || exp[i] = ']'){
    if (top = = -1 || ! is matchingpair (pop(), exp[i])){
     Printf ("not balance expression \n");
        return 0;
        }
    }
  }
  //check stack empty
  if (top = = -1)
        Printf ("balanced expression \n");
    else
      Printf ("Not balanced expression \n");
      return 0;
  }
```

Output:

enter expression = (A+B)*((-p)}

Balanced expression

Q3. Next greater element

Algorithm

1. start
2. Take array input
3. Transverse array
4. Use stack t track elements
5. for each element:
   - compare with stack top
   - print next greater
6. stop

C program

```
# include <stdio.h>
# def MAX 100
    int stack[MAX];
    int top = -1
  // push element
    void push(int x){
    stack[++top] = x
    }
  // pop element
    int pop() {
    return stack [top--];
    }
int main(){
  int arr[] = {4,5,2,10,8};
    int n = 5, i;
    push (arr[0]);
  for (i = 1; i < n; i++){
    // find next greater
  while (top != -1 && stack[top] < arr[i]){
        Printf ("%d -> %d\n", pop(), arr[i]);
        }
        push (arr[i]);
        }
```

```
//remaining elements
while (top != -1){
        printf("%d -> -1 \n", pop());
}
return 0;
}
```

output: 4->5, 5->10, 2->10, 10->-1, 8->-1

Q4. print queue simulation

algorithm:

1) start
2) intialize queue
3) display menu:
   - add document
   - print document
   - display queue
4. perform operations using FIFO
5. stop

program

```
#include <stdio.h>
# define MAX 5
int queue [MAX];
int front = -1, rear = -1;
//insert element
void enqueue (int x){
  if (rear == MAX -1){
      printf ("Queue FULL\n");
      return;
  }
  if (front == -1) front = 0;
  queue (++rear) = x;
}
delete element
void dequeue (){
  if (front == -1){
  printf ("Queue empty\n");
```

```
    return;
  }
Printf("printed document: %d\n", queue[front++]);
  if(front > rear) front = rear = -1;
}
// display queue
void display() {
  int i;
  if (front == -1){
  Print("No pending document\n");
   return;
  }
Printf("Pending documents:");
  for (i=front; i<=rear; i++)
    printf("%d", queue[i]);
  Print+("\n");
}int main() {
  int choice,doc;
  while (1){
  Printf("\n1. Add 2. print 3. display 4. exit\n");
  scanf("%d", &choice);
  switch(choice){
  case 1:
  Printf("enter document id:");
   scanf("%d", &doc);
   enqueue(doc);
   break;
  case 2:
  dequeue();
   break;
  case 3:
     display();
      break;
   case 4:
       return 0;
  }
}
```

}

output:

1. add 2. print 3. Display 4. exit

1.

enter document id: 101

1. Add 2. print 3. display 4. exit

3

pending documents: 101